AARHUS UNIVERSITY
DEPARTMENT OF ELECTRICAL AND COMPUTER ENGINEERING

Term:	FALL 2023
Examination:	Declarative Programming
Date:	05 January 2024
Duration:	3 hours

- This is the exam for the lecture Declarative Programming. It consists of 18 questions. You have 3 hours to answer all of them. Strategically, you should first scan the exam paper quickly in its entirety, and identify the groups of questions that seem easiest to you. ***You should answer those first*** and then move on to the others.
- For each question it is indicated how and where the answer must be provided. There are questions that require writing a calculation or the result of a calculation, and questions that require you to write Ocaml or Prolog code, in particular. Only what is written or drawn inside the specified areas is considered as your answer.
- **If you are in doubt about what is expected, please ask!**
- Each question has a weight **[w]**, where **w** is an integer number, specifying the number of points awarded for that question. You can find the weight at the top right corner of each question. The total number of points of all questions together is 100.

Question 1 **[6]**

Recall the signature of the functions (=) and (<)

```
val (=) : 'a -> 'a -> bool
val (<) : 'a -> 'a -> bool
```

What are the signatures of the functions `f`, `g` and `h`?

```
let f x y = x
let g x y = x < x
let h x y z = z = x && x < y && y < z
```

Answer:

`val f :` 'a -> 'b -> 'a

`val g :` 'a -> 'b -> bool

`val h :` 'a -> 'a -> 'a -> bool

Question 2 **[4]**

(a) Define a function with signature `val bigger : float -> float -> float` that yields the maximum of two input values, e.g.,

```
5.0 = bigger 1.0 5.0
0.5 = bigger 0.5 0.25
1.0 = bigger 1.0 -1.0
```

Answer: `let` bigger x y = if x +. 0. < y then y else x

(b) Define a function with signature

```
val parabola : (int -> int -> int) -> int -> int
```

that takes three inputs a, b, c and returns a function that takes one input x and yields $ax^2 + bx + c$, e.g.:

```
13 = (parabola 2 1 3) 2
12 = (parabola 0 1 2) 10
```

Answer: `let` parabola a b c = fun x -> a*x*x + b*x + c

Question 3 **[6]**

Complete the following recursive function `eval es stk` so that it implements the following behaviour:

```
eval [] [false; true] = [false; true]
eval ["true"] [] = [true]
eval ["true"; "false"; "and"] [] = [false]
eval ["true"; "true"; "and"; "false"; "and"] [] = [false]
```

```
let rec eval es stk =
  match es, stk with
    [], stk                                  -> stk
    | "true"::es', stk                       -> eval es' (true::stk)
    | "false"::es', stk                      -> eval es' (false::stk)
    | "and"::es', p::q::stk                  -> eval es' ((p && q)::stk)
```

Question 4 **[6]**

You are given the following tail recursive implementation of function `checkseq n` that extracts the list of digits occurring in the decimal representation of `n`. E.g.,

```
checkseq 56 = [5; 6]
checkseq 567 = [5; 6; 7]
```

```
let rec checkseq n =
  if n = 0
    then []
    else checkseq (n / 10) @ [n mod 10]
```

Complete the tail recursive function `checkseq_tr n ac` below that behaves just as the function `checkseq n`. Hint: The argument `ac` is the accumulator.

```
let rec checkseq_tr n ac =
  if n = 0

    then ac

    else checkseq_tr (n / 10) ((n mod 10)::ac)
```

Question 5 **[4]**

Functions `mapzip`, `atleast`, `(@@)` and `min` are defined as

```
let rec mapzip f xys =
  match xys with
    (x::xs', y::ys') ->
      f x y::mapzip f (xs', ys')
  | (_, _) -> []

let rec atleast n xs =
  match xs with
    [] -> []
  | (x::xs') -> if x >= n
    then x::atleast n xs'
    else atleast n xs'

let (@@) f g x = f (g x)

let min x y : int = if x < y then x else y
```

(a) Using functions `mapzip` and `min`, implement function `minmapzip ([x1; x2; ...], [y1; y2; ...])` that returns the list of minima `[z1; z2; ...]` corresponding to the pairs `(x1, y1)`, `(x2, y2)`, `...`.
Answer:

```
let minmapzip = mapzip min
```

(b) Using functions `mapzip`, `atleast` and `(@@)`, implement a function `bndmapzip ([x1; x2; ...], [y1; y2; ...])` that returns the list of sums `[z1; z2; ...]` of the pairs `(x1, y1)`, `(x2, y2)`, `...` that are at least `0`. Hint: Use the function `(+)` corresponding to the addition operator `+`.
Answer:

```
let bndmapzip = mapzip ((@@) (atleast 0) (+))
```

Question 6 **[7]**

(a) Unnest the following function `et`

```
let et w =
  let tu s =
    let brute j c = j * c + s in
    brute w w - s in
  tu 1
```

Answer:

```
let brute  j c s = j * c + s
let tu     s w = brute w w s - s
let et     w = tu 1 w
```

(b) Unnest the following function `itsy`

```
let itsy y p b =
  let bitsy x = y b x in
  let teenie q = bitsy q in
  let weenie z = teenie (z p) in
  weenie (fun x -> x)
```

Answer:

```
let bitsy   x y b = y b x
let teenie  q y b = bitsy q y b
let weenie  z p y b = teenie (z p) y b
let itsy    y p b = weenie (fun x -> x) p y b
```

Hint: Function declarations can be nested, that is, “inner” functions are declared locally to “outer” functions. Parameters to outer functions can be referred to in inner functions.
Nested functions can be unnested by declaring all “inner” functions globally before the outer function making the passed parameters explicit. The behaviours of the nested and corresponding unnested functions are indistinguishable.

Question 7 **[5]**

Given the type `nat` below

```
type nat = Z | S of nat
```

and the two functions `halve` and `count`

```
let rec halve n =
  match n with
    S (S m) -> Z::halve m
  | _ -> []

let rec count xs =
  match xs with
    _::xs' -> S (count xs')
  | [] -> Z
```

We can compute

```
count (halve (S (S (S (S (S Z)))))) = S (S Z)
```

Reading the function definitions as a system of equations as we have done in the course, calculate the result of the computation by inserting the missing steps:

```
count (halve (S (S (S (S (S Z))))))
```

= count (Z::(halve (S (S (S Z)))))

= count (Z::Z::(halve (S Z)))

= count (Z::Z::[])

= S (count Z::[])

= S (S (count []))

= S (S Z)

Question 8 **[5]**

(a) Using the definition of the functions `map` given by

```
let rec map f xs =
  match xs with
    x::xs' -> f x::map f xs'
  | [] -> []
```

implement a function `fliplist` that "flips" the pairs of lists as follows

```
[] = fliplist []
[(2, 1)] = fliplist [(1, 2)]
[(4, 3); (1, 2)] = fliplist [(3, 4); (2, 1)]
```

Answer:

```
let fliplist = map (fun (x, y) -> (y, x))
```

Note: You are not allowed to add parameters before the `=`.
Hint: Use a lambda abstraction to flip pairs.

(b) Using function `split` (that transforms a list of pairs into a pair of lists) in addition where

```
let rec split xs =
  match xs with
    (y, z)::xs' -> let (ys, zs) = split xs' in (y::ys, z::zs)
  | [] -> ([], [])
```

implement a function `splitfliplist` that adds all integer values of a list of lists, that is,

```
([], []) = splitfliplist []
([2], [1]) = splitfliplist [(1, 2)]
([2; 4], [1; 3]) = splitfliplist [(1, 2); (3, 4)]
([2; 4; 6], [1; 3; 5])
    = splitfliplist [(1, 2); (3, 4); (5, 6)]
([4; 1], [3; 2]) = splitfliplist [(3, 4); (2, 1)]
```

Answer:

```
let splitfliplist = split fliplist
```

Note: You are not allowed to add parameters before the `=`.

Question 9 **[8]**

Given the types

```
type gs = G | P of gs | S of hs
 and hs = H | Q of hs | T of gs
```

the following functions `ginv` and `hinv` "invert" values of types `gs` and `hs`.

```
let rec ginv gs =
    match gs with
      G -> H
    | P x -> Q (ginv x)
    | S x -> T (hinv x)
and hinv hs =
    match hs with
      H -> G
    | Q x -> P (hinv x)
    | T x -> S (ginv x)
```

Considering the function definitions as a system of equations, calculate:

(a) The value of `hinv (Q (Q H))`

`hinv (Q (Q H))`

= P (hinv (Q H))

= P (P (hinv H))

= P (P (G))

(b) The value of `ginv (S (Q (T (P G))))`

`ginv (S (Q (T (P G))))`

= T (hinv (Q (T (P G))))

= T (P (hinv (T (P G))))

= T (P (S (ginv (P G))))

= T (P (S (Q (ginv G))))

= T (P (S (Q (H))))

Question 10 **[6]**

Consider the pattern matching algorithm match from the course

$$
\begin{array}{lll}
\mathrm{match}\ S\ u\ =\ u & & \text{if } S = \emptyset \\
\qquad\qquad \mathrm{match}\ (S \setminus (x,t))\ u & & \text{if } (x,t) \in S \cap u \\
\qquad\qquad \mathrm{match}\ (S \setminus (x,t))\ (u \cup \{(x,t)\}) & & \text{if } (x,t) \in S \wedge (x,s) \notin u \text{ for all } s \\
\qquad\qquad \mathrm{match}\ (S \cup \{(s_1,s'_1),\ldots,(s_k,s'_k)\}\ u & & \text{if } (c(s_1,...,s_k), c'(s'_1,...,s'_k)) \in S \wedge c = c' \\
\qquad\qquad \mathbf{fail} & & \text{otherwise}
\end{array}
$$

Let a, b, c and x denote variables and G, S, T, U and Z constants. As usual, we leave away empty argument lists, e.g., instead of $Z()$ we write Z.

Calculate:

(a) The value of $\mathrm{match}\ \{(T(S(a),S(S(b))),\ T(S(S(Z)),S(S(G(c,U)))))\}\ \emptyset$

$\mathrm{match}\ \{(T(S(a),S(S(b))),\ T(S(S(Z)),S(S(G(c,U)))))\}\ \emptyset$

$=$ match {(S(a),S(S(Z))), (S(S(b)),S(S(G(c,U))))} Ø

$=$ match {(a,S(Z)), (S(S(b)),S(S(G(c,U))))} Ø

$=$ match {(a,S(Z)), (S(b),S(G(c,U)))} Ø

$=$ match {(a,S(Z)), (b,G(c,U))} Ø

$=$ match {(b,G(c,U))} {(a,S(Z))}

$=$ match {} {(a,S(Z)), (b,G(c,U))}

$=$ {(a,S(Z)), (b,G(c,U))}

(b) The value of $\mathrm{match}\ \{(U(G(a,b),a),\ U(G(x,Z),U(x)))\}\ \emptyset$

$\mathrm{match}\ \{(U(G(a,b),a),\ U(G(x,Z),U(x)))\}\ \emptyset$

$=$ match {(G(a,b),G(x,Z)), (a,U(x))} Ø

$=$ match {(a,x), (b,Z), (a,U(x))} Ø

$=$ match {(b,Z), (a,U(x))} {(a,x)}

$=$ match {(a,U(x))} {(a,x), (b,Z)}

$=$ fail

Question 11 **[5]**

Using the truth table method, determine whether the formula

$$P \Rightarrow (Q \Rightarrow (\neg P \Rightarrow \neg Q))$$

is satisfiable.

Fill in the truth table:

P	Q	$P \Rightarrow (Q \Rightarrow (\neg P \Rightarrow \neg Q))$
T	T	T
T	F	T
F	T	T
F	F	T

Tick the correct answer:

☒ The formula is satisfiable.

☐ The formula is unsatisfiable.

Question 12 **[4]**

Determine whether the formula

$$\begin{array}{ll} & \neg U \vee \neg A \vee \neg H \\ \wedge & U \vee A \vee H \\ \wedge & \neg U \vee A \vee H \\ \wedge & U \vee \neg H \end{array}$$

is satisfiable applying the DPLL algorithm, using the rules

(UR) the 1-literal rule (aka unit clause rule),
(PR) the affirmative-negative rule (aka pure literal rule),
(SR) the splitting rule.

Fill into the boxes the formulas resulting from the rule applications indicated between the boxes.

1:

$$\begin{array}{ll} & \neg U \vee \neg A \vee \neg H \\ \wedge & U \vee A \vee H \\ \wedge & \neg U \vee A \vee H \\ \wedge & U \vee \neg H \end{array}$$

Applying rule **(SR)** choosing $H = \top$ to the formula in box 1 yields:

2:

¬U ¬A U

Applying rule **(UR)** to the formula in box 2 yields:

3:

¬A

Applying rule **(UR)** to the formula in box 3 yields:

4:

T

Tick the correct answer:
☒ The formula is satisfiable.
☐ The formula is unsatisfiable.

Question 13 **[6]**

The following table specifies two lambda terms in the first row. Complete the empty cells in the remaining rows of the table according to the lambda terms for each column.
Remark: In your answers you may write "L" to represent "λ" if you prefer.

Lambda term	$\lambda f\ g\ h \cdot ((\lambda x \cdot f\ (x\ x))\ (\lambda x \cdot g\ (x\ y)))$	$(\lambda x \cdot v)(\lambda v \cdot x)$
Name the **type** of lambda term	combination	combination
Rewrite the term in its fully curried form	Lf · (Lg · (Lh ·((Lx · f (x x)) (Lx · g (x y)))))	(Lx · v)(Lv · x)
Specify the **free** variables	y	v, x
Specify the **bound** variables	f, g, h, x	x, v

Question 14 **[8]**

King Graham rules over a country called Daventry. He is on a quest to find treasure that is hidden somewhere in his kingdom. Consider the following Prolog program:

```
location(castle).
location(goat_fields).
location(garden_path).
location(cottage).
location(troll_bridge).
location(old_well).
location(witch_cave).

signpost(witch_cave, cottage).
signpost(castle, goat_fields).
signpost(goat_fields, garden_path).
signpost(garden_path, witch_cave).
signpost(old_well, troll_bridge).
signpost(castle, garden_path).
signpost(troll_bridge, cottage).
signpost(garden_path, old_well).

at(treasure, cottage).
```

(a) For each of the following Prolog queries applied to the above program, write down the first query result when executed in Prolog. For example:

```
?- location(L).
```

First query result when executed in Prolog:

```
L = castle.
```

(i) `?- signpost(garden_path, L).`

First query result when executed in Prolog:

L = witch_cave.

(ii) `?- location(Loc), at(Item, Loc).`

First query result when executed in Prolog:

Loc = cottage, Item = treasure.

(b) A *path* is a list of locations that King Graham can visit in sequence, e.g.:
`[garden_path, old_well, troll_bridge]`

Hint: To answer the following questions you may need to use Prolog built-in predicates: *member/2*, *length/2*, *append/3*, *reverse/2*, *not/1*, and the list terms *[Head | Tail]* and *[]*.

(i) If King Graham goes to the witch's cave, he'll get eaten. Write a Prolog rule for the predicate *eaten_by/2* that is True if King Graham gets eaten by the witch on a given path *Path*:

```
eaten_by(witch, Path) :- member(witch_cave, Path).
```

(ii) Write a Prolog rule for the predicate *has_treasure/1* that is True if the given path *Path* has a location with treasure:

```
has_treasure(Path) :- at(treasure, Location), member(Location, Path).
```

(iii) If King Graham visits the goat fields, a goat will follow him for the rest of his journey. If King Graham goes to the troll bridge without a goat (i.e. without first visiting the goat fields), he'll get eaten by the troll.
Write a Prolog rule for the predicate *eaten_by/2* that is True if King Graham gets eaten by the troll on a given path *Path*:

```
eaten_by(troll, Path) :-
```

(c) King Graham can only follow sign posts (or he gets lost), so there must be a sign post from each location to the next location in the path. Define a predicate *path/1* that is True when the argument is a list of locations such that there is a sign post showing King Graham the way between each location and its successor location in the list. E.g. the following query should return True:

```
?- path([garden_path, old_well, troll_bridge]).
true.
```

The following query should return False because there is no sign post at *old_well* that shows the way back to *garden_path*:

```
?- path([garden_path, old_well, garden_path]).
false.
```

(i) Define the predicate *path/1* with Prolog rules using *recursion*.

Hint: you'll need to define a base case rule and a recursive case rule.

```
path(P) :- ____________________________________________
____________________________________________________
path(P) :- ____________________________________________
____________________________________________________
____________________________________________________
```

(ii) Write a query that generates a path *P* using the *path/1* predicate. The path should start from the castle, King Graham must find the treasure, and he must not get eaten by anything.

```
?- path(P), ___________________________________________
____________________________________________________
____________________________________________________
```

Question 15 **[4]**

Consider the following Prolog program that defines two predicates $p/4$ and $q/2$.

```
p(_,[],[],[]).
p(P, [H|T], [H|A], B)      :- H < P,  p(P,T,A,B).
p(P, [H|T], A,     [H|B]) :- H >= P, p(P,T,A,B).

q([],[]).
q([P|T],S) :-
  p(P,T,A,B),
  q(A,SA),
  q(B,SB),
  append(SA,[P|SB],S).
```

(a) Write down the result of each of the following unification queries.

```
?- p(5,[7,3],A0,B0) = p(P,[H|T],A,[H|B]).
```

First query result when executed in Prolog:

P = 5, H = 7, T = [3], A0 = A, B0 = [7|B].

```
?- q([5,2,7,3],S0) = q([P|T],S).
```

First query result when executed in Prolog:

P = 5, T = [2,7,3], S0 = S.

(b) For each of the following Prolog queries applied to the above program, write down the first query result when executed in Prolog.

```
?- p(5,[7,3],A0,B0).
```

First query result when executed in Prolog:

A0 = [3], B0 = [7].

```
?- q([5,2,7,3],S).
```

First query result when executed in Prolog:

S = [5,7,2,3].

Question 16 **[6]**

(a) Given two natural numbers a, b we can define the remainder of dividing a by b recursively as follows:

- If $a < b$ then the remainder of a/b is a.
- Otherwise, the remainder of a/b is the remainder of $(a - b)/b$.

For example:

- The remainder of dividing 10 by 5 is 0.
- The remainder of dividing 21 by 8 is 5.

Use this definition to implement the predicate $rem/3$ such that $rem(A, B, R)$ is True when natural number A divided by natural number B has remainder R. Write recursive Prolog rules to define the predicate $rem/3$.
Hint: You will need to implement a Prolog fact or rule for the base case, and a Prolog rule for the recursive case.
Remark: You can use Prolog's built-in *is/2* operator for arithmetic.

Answer:

(b) The greatest common divisor (*gcd*) of two natural numbers a, b is the largest natural number that divides both a and b.
For example:

- The *gcd* of 10 and 5 is 5.
- The *gcd* of 18 and 27 is 9.

The *gcd* between two natural numbers can be defined recursively as follows:

- The gcd of a and 0 is a.
- Otherwise, the *gcd* of a and b is the *gcd* of b and the remainder of a/b.

Use this definition to implement the predicate $gcd/3$ such that $gcd(A, B, G)$ is True when G is the *gcd* of natural numbers A and B. Write recursive Prolog rules to define the predicate $gcd/3$.
Hint: You will need to implement a Prolog fact or rule for the base case, and a Prolog rule for the recursive case.

Answer:

Question 17 **[4]**

In the lectures we discussed the unification procedure, consisting of six rewriting "rules" that step-by-step transform a set of unification expressions until arriving at a "solved substitution" or a failure.

The six rules are: *swap*, *eliminate*, *decompose*, *conflict*, *occurs*, *delete*.

Apply the same procedure to unify the expressions in (a) and (b) below. Apply exactly one "rule" at each step, and specify the rule used by writing the rule name to the right side of the new unification expression set (as we did in lectures).
Remark: In your answers you can write "$=$" to represent "$\approx$" if your prefer.

(a) Unify the term `c(X, c(d, c(c(X, d), Y)))` with the term `c(c(Z, Y), c(Y, c(Z, d)))`.

Answer:

$\{c(X, c(d, c(c(X, d), Y))) \approx c(c(Z, Y), c(Y, c(Z, d)))\}$

→ {X c(Z,Y), c(d,c(c(X,d),Y)) c(Y,c(Z,d))} decompose

{X c(Z,Y), d Y, c(c(X,d),Y)) c(Z,d)} decompose

{X c(Z,Y), d Y, c(X,d) Z,Y d} decompose

{X c(Z,Y), d Y, c(c(Z,Y),d) Z,Y d} eliminate

fail occurs

(b) Unify the term `q(X,7,X,7,w(X,Y))` with the term `q(7,Y,7,X,w(7,X))`.

Answer:

$\{q(X, 7, X, 7, w(X, Y)) \approx q(7, Y, 7, X, w(7, X))\}$

→ {X 7, 7 Y, 7 X, w(X, Y) w(7, X)} decompose

{X 7, 7 Y, 7 X, Y X} decompose

{X 7, 7 Y, 7 7, Y 7} eliminate

{X 7, 7 Y, Y 7} delete

{X 7, Y 7} swap

{X 7, Y 7}

Question 18 **[6]**

Consider the following Prolog program that uses the built-in arithmetic operator *is*:

```
x(point(X,_), X).
y(point(_,Y), Y).

translate(P, V, point(X0,Y0)) :-
 x(P,X),  y(P,Y),
 x(V,DX), y(V,DY),
 X0 is X + DX,
 Y0 is Y + DY.
```

(a) For each of the following Prolog queries applied to the above program, write down the first query result when executed in Prolog, or write down the exception that is thrown.

(i) `?- translate(point(7,19), point(3,4), P).`

First query result or exception when executed in Prolog:

_____

(ii) `?- translate(point(X0,19), point(3,Y0), point(0,0)).`

First query result or exception when executed in Prolog:

_____

(iii) `?- translate(point(1,3), point(2,2), point(X,1)).`

First query result or exception when executed in Prolog:

_____

(b) Rewrite the definition of the $translate/3$ predicate so that CLP(FD) arithmetic is used instead of Prolog's built-in arithmetic operator.

Answer:

```
translate(P, V, point(X0,Y0)) :-
```

_____

_____

_____

_____

_____

_____

(c) Are the results of any of the queries in question (a) different when using the new CLP(FD) implementation of the $translate/3$ predicate?

If yes, write the question number (i, ii, or iii) in the box below, or write "None" if the query results are all the same.

The new query result is: